

ASSIGNMENT SUBMISSION

Mini AI Support Ticket System – Mindtide.ai Full Stack Developer Position

GITHUB REPOSITORY

https://github.com/21mis7174/support_ticket_system

LIVE DEMO

Frontend: <http://13.233.109.44:3000>

Backend: <http://13.233.109.44:8000>

API Docs: <http://13.233.109.44:8000/docs>

PROJECT OVERVIEW

Full-stack web application for managing support tickets. Users can create tickets with priority levels, view all tickets, filter by status, and mark tickets as resolved with real-time updates.

KEY FEATURES

- Create tickets with title, description, and priority (Low/Medium/High)
- View all tickets in organized list format
- Filter tickets by status (All / Open / Resolved)
- Mark tickets as resolved with automatic timestamp
- Real-time UI updates and status indicators
- Responsive design for desktop and mobile
- Production-ready Docker deployment on AWS EC2

TECHNOLOGY STACK

Backend: Python 3.11, FastAPI, Pydantic, Motor (async MongoDB), Uvicorn

Frontend: Next.js 16, TypeScript, React 19, Tailwind CSS
Database: MongoDB 7.0
DevOps: Docker, Docker Compose, AWS EC2 (t2.micro, ap-south-1)

API ENDPOINTS

1. CREATE TICKET (POST /tickets)

```
curl -X POST http://13.233.109.44:8000/tickets \  
-H "Content-Type: application/json" \  
-d '{  
  "title": "Login page broken",  
  "description": "Users cannot log in after deployment",  
  "priority": "high"  
}
```

Response: 201 Created with ticket object

2. LIST ALL TICKETS (GET /tickets)

```
curl http://13.233.109.44:8000/tickets
```

Response: 200 OK with array of tickets

3. RESOLVE TICKET (PATCH /tickets/{id}/resolve)

```
curl -X PATCH http://13.233.109.44:8000/tickets/507f1f77bcf86cd799439011/resolve
```

Response: 200 OK with updated ticket

ERROR RESPONSES

Status 422: Missing required fields

Status 404: Ticket not found

Status 400: Invalid ticket ID

QUICK START

OPTION 1: Docker Compose (Recommended)

```
git clone https://github.com/21mis7174/support_ticket_system.git
cd support_ticket_system
docker compose up -d
```

Access:

- Frontend: <http://localhost:3000>
- Backend: <http://localhost:8000>
- API Docs: <http://localhost:8000/docs>

OPTION 2: Local Development

BACKEND:

```
cd backend/
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python run.py
```

FRONTEND (new terminal):

```
cd frontend/
npm install
npm run dev
```

ENVIRONMENT CONFIGURATION

Backend (.env):

```
MONGODB_URL=mongodb://mongo:27017/support_tickets_db
DATABASE_NAME=support_tickets_db
```

Frontend (.env.local):

```
NEXT_PUBLIC_API_URL=http://13.233.109.44:8000
```

AWS EC2 DEPLOYMENT

Instance Type: t2.micro (Free tier eligible)

Region: ap-south-1 (Mumbai)

AMI: Amazon Linux 2023

Public IP: 13.233.109.44

Hostname: ec2-13-233-109-44.ap-south-1.compute.amazonaws.com

Services Running:

- MongoDB 7.0 (Port 27017)
- FastAPI Backend (Port 8000)
- Next.js Frontend (Port 3000)

TESTING THE APPLICATION

Visit <http://13.233.109.44:3000> to:

1. Create a new ticket
 - Fill title, description, priority
 - Submit and verify in list
2. View all tickets
 - 10 seeded test tickets visible
 - Display title, description, priority, status
3. Filter tickets by status
 - All, Open, Resolved tabs
4. Resolve a ticket
 - Click "Mark as Resolved" button
 - Status updates in real-time
5. Test API endpoints
 - Visit <http://13.233.109.44:8000/docs>

ASSIGNMENT COMPLETION STATUS

PART 1: BACKEND (FastAPI) ✓

- ✓ POST /tickets endpoint
- ✓ GET /tickets endpoint
- ✓ PATCH /tickets/{id}/resolve
- ✓ MongoDB integration with Motor
- ✓ Pydantic models and validation
- ✓ Error handling (422/404/400)
- ✓ Clean folder structure

PART 2: FRONTEND (Next.js) ✓

- ✓ Create Ticket page with form
- ✓ Tickets List page
- ✓ Resolve button functionality
- ✓ Filter by status (All/Open/Resolved)
- ✓ Fetch API integration
- ✓ Tailwind CSS styling
- ✓ Loading and error states

PART 3: DOCKER & CONTAINERIZATION ✓

- ✓ Backend Dockerfile (multi-stage)
- ✓ Frontend Dockerfile (multi-stage)
- ✓ docker-compose.yml orchestration

- ✓ Health checks for all services
- ✓ Non-root user execution
- ✓ Service dependency management

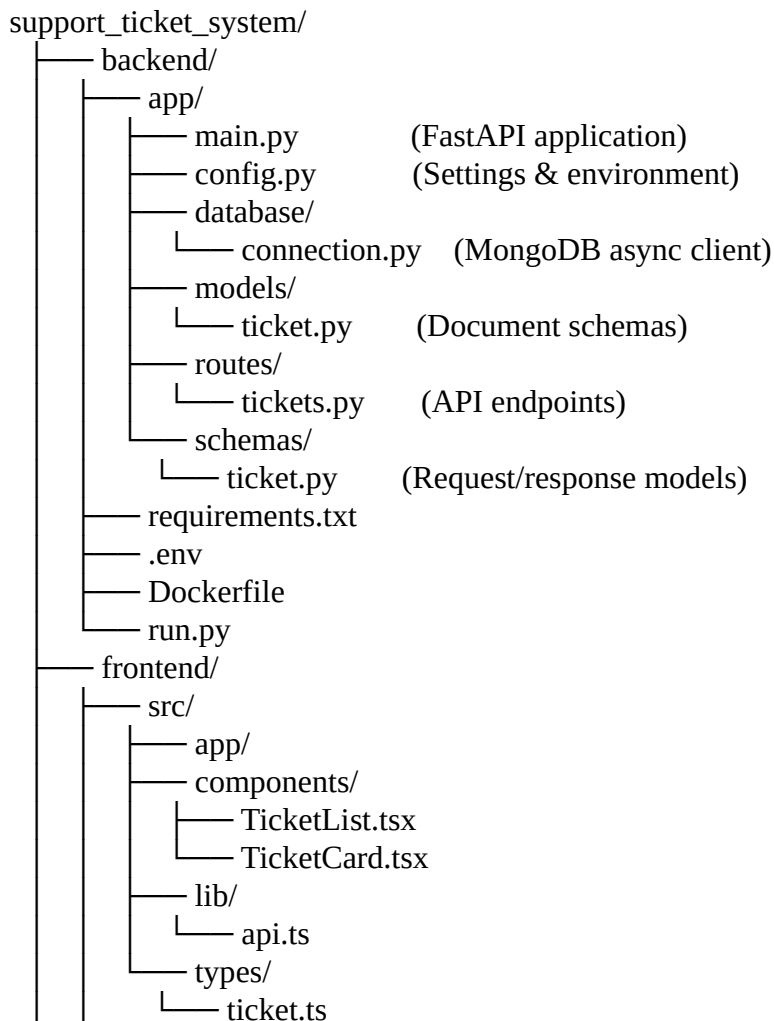
PART 4: CLOUD DEPLOYMENT ✓

- ✓ AWS EC2 instance (t2.micro, ap-south-1)
- ✓ Public IP: 13.233.109.44
- ✓ MongoDB with 10 seeded test tickets
- ✓ CORS configuration
- ✓ Auto-restart on failure
- ✓ UTC timezone handling

PART 5: SUBMISSION REQUIREMENTS ✓

- ✓ GitHub repository link
- ✓ README with setup instructions
- ✓ API endpoints documentation
- ✓ Screenshots (home_page.png, new_ticket.png)
- ✓ Live demo link (running 24/7)

REPOSITORY STRUCTURE



```
|
|   ├── Dockerfile
|   ├── package.json
|   └── next.config.ts
|   └── screenshots/
|       ├── home_page.png
|       └── new_ticket.png
|   ├── docker-compose.yml
|   ├── README.md
|   └── ASSIGNMENT_SUBMISSION.txt
```

APPLICATION SCREENSHOTS

Figure 1: Main Tickets List Page

- Displays all support tickets
- Status indicators (Open/Resolved)
- Priority badges (Low/Medium/High)
- Relative timestamps
- Filter tabs (All/Open/Resolved)
- Mark as Resolved buttons

Figure 2: Create New Ticket Form

- Title input field
- Description textarea
- Priority dropdown (Low/Medium/High)
- Submit button
- Input validation

TROUBLESHOOTING

Frontend not connecting to backend:

- Check NEXT_PUBLIC_API_URL in .env.local
- Check browser console (F12) for CORS errors
- Verify backend: `curl http://13.233.109.44:8000/health`

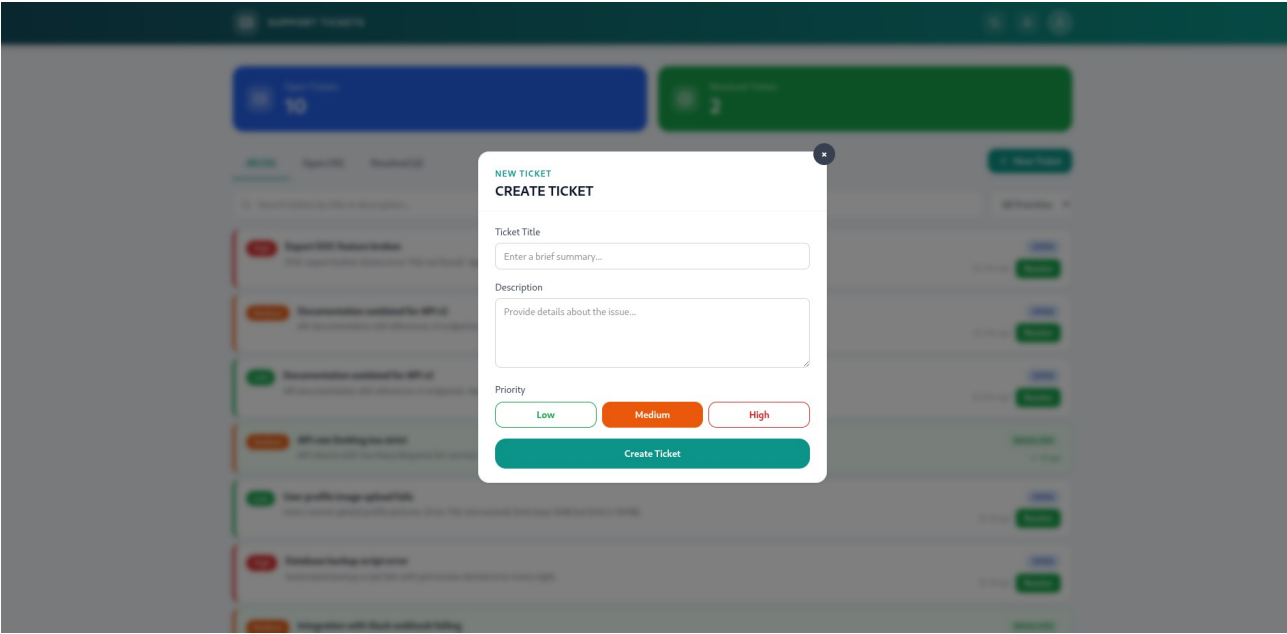
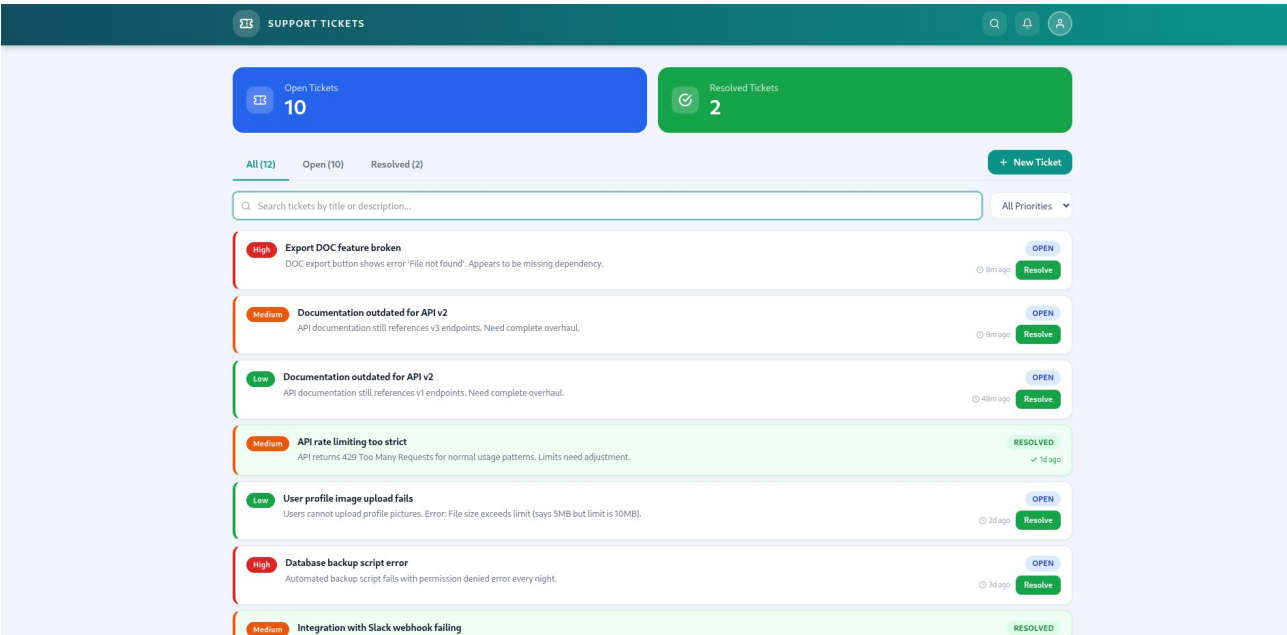
MongoDB connection issues:

- Check container: `docker ps | grep mongo`
- View logs: `docker logs support_tickets_mongo`
- Connect: `docker exec -it support_tickets_mongo mongosh`

Tickets not showing:

- Check database: `docker exec mongo mongosh`
- Count tickets: `db.support_tickets_db.tickets.countDocuments()`
- Check API: `curl http://13.233.109.44:8000/tickets`

SCREENSHOTS:



SUBMISSION CHECKLIST

GitHub Repository: https://github.com/21mis7174/support_ticket_system
Live Demo: <http://13.233.109.44:3000>
Backend API: <http://13.233.109.44:8000>
API Documentation: <http://13.233.109.44:8000/docs>
Complete source code: GitHub
README.md: Comprehensive documentation
Screenshots: 2 pages (home_page.png, new_ticket.png)
Docker config: docker-compose.yml
All 5 parts: Implemented and tested